

Bericht DS01

In den Aufgaben des Übungsblattes ging es darum, eine verteilte Anwendung zu implementieren. Konkret sollte mit UDP-Multicast ein logischer Ring umgesetzt werden, in dem ein Token zwischen den Prozessen weitergegeben wird. In Aufgabe 03 wurde statt UDP der Simulator verwendet. Dieser simuliert das Senden und Empfangen von Nachrichten sowie den Multicast. Jeder Prozess, also jeder Knoten, zündet mit einer Wahrscheinlichkeit p eine Rakete. Nach jeder Runde wird diese Wahrscheinlichkeit p kleiner. Jede gezündete Rakete wird anschließend an alle anderen Prozesse verschickt. Die Anwendung endet nach k leeren Runden. Da in der Aufgabenstellung keine konkreten Werte für p und k vorgegeben wurden, wurden für alle Aufgaben dieselben Werte gewählt: $p = 1$ als Startwert und $k = 3$ für die Terminierung nach drei leeren Runden. Dadurch zündet in der ersten Runde jeder Knoten eine Rakete. Die einheitlichen Werte sorgen dafür, dass die Ergebnisse der Aufgaben besser vergleichbar sind. In Aufgabe 04 ging es anschließend darum, mögliche Konsistenzprobleme zu erkennen und, wenn möglich, zu vermeiden.

Implementierung Aufgaben 01-03

Da es in der Aufgabenstellung keine konkreten Vorgaben zur Programmiersprache gab, wurden die Aufgaben 01 und 02 in Python umgesetzt. Der Implementierungsaufwand des Grundgerüsts war dadurch geringer als bei einer Java-Umsetzung, der eigentliche Aufwand lag im Verständnis der Socket- und Multicast-Programmierung. Jeder Knoten des Rings läuft dabei als eigener Prozess auf demselben Testrechner. Für die Weitergabe des Tokens wird ein UDP-Socket verwendet. Beim Start wird über die ProzessID festgelegt, welche Position der Prozess im Ring einnimmt. Daraus berechnet sich auch der UDP-Port ($\text{base_port} + \text{pid}$), unter dem der Prozess erreichbar ist. Das Token selbst, was weitergegeben wird, ist eine JSON-Nachricht mit drei Feldern: einem Hop-Zähler, der Zahl bisheriger leerer Runden, und einem Flag, ob in der aktuellen Runde bereits jemand gezündet hat. Für die Raketennachrichten tritt jeder Prozess derselben Multicast-Gruppe bei und kann so die Zündungen aller anderen empfangen. Eine Rakete wird mit der Nachricht ROCKET per Multicast verschickt. Prozess 0 nimmt eine Sonderrolle ein, denn er erkennt beim Empfang des Tokens, dass eine neue Runde gestartet ist, misst die Rundenzeiten, zählt die leeren Runden und sendet nach Erreichen von k leeren Runden eine TERMINATE-Nachricht in den Ring, die jeder Knoten an seinen Nachfolger weiterleitet. Am Ende schreibt jeder Prozess seine Statistiken (Anzahl gezündeter Raketen, bei Prozess 0 zusätzlich die Rundenzeiten) als JSON-Datei.

Aufgabe 02 baut auf der Logik von Aufgabe 01 auf, der Implementierungsaufwand war daher gering. Im Gegensatz zu Aufgabe 01, wo die Prozesse auf dem localhost gelaufen sind, wurde hier jeder Prozess auf einem realen Computer gestartet. Deswegen wurde eine zusätzliche Konfigurationsdatei benötigt, die alle Prozesse mit Host und Port auflistet, damit jeder Prozess seinen Nachfolger im Ring kennt. Grundsätzlich bleibt der Kommunikationsablauf gleich. Falls Multicast in der Netzwerkumgebung nicht unterstützt wird, wurde als Fallback ein Unicast eingebaut.

Aufgabe 03 zu implementieren war aus zwei Gründen relativ einfach. Erstens konnte die Logik von den ersten Aufgaben einfach auf den Simulator übertragen werden. Zweitens war

eine Implementierung der Netzwerkfunktionen nicht mehr nötig, da der Simulator die Funktionen `send()`, `receive()` und `broadcast()` bereits mitbringt. Als Ausgangspunkt wurde der Beispieltest `OneRingToRuleThemAllTest` verwendet, der schon den grundlegenden Aufbau eines Token Rings zeigt. Für die Kommunikation zwischen den Knoten werden drei Nachrichtentypen verwendet: Token, Firework und Terminate. Der wesentliche Unterschied zu Aufgabe 01 besteht darin, dass das Token als typisiertes Java Objekt statt als JSON-Nachricht verwendet wird und die Kommunikation nicht mehr über selbst implementierte UDP Sockets, sondern über die Methoden des Simulators erfolgt.

Vergleich Aufgaben 01–03

Vergleicht man die Ergebnisse, fällt auf, dass die Anzahl der abgeschlossenen Runden, die Anzahl der gezündeten Multicasts und die Zeiten bei gleichem n relativ ähnlich sind. Kleinere Abweichungen sind dabei normal, weil der Zufall durch die Zündwahrscheinlichkeit p weiterhin eine Rolle spielt. Der größte Unterschied zeigt sich beim maximal erreichbaren n .

Aufgabe	Realisierung	Max n
1	UDP localhost	485
2	UDP verteilte Rechner	3
3	Sim4da Simulator	16384

In Aufgabe 01, bei der alle Prozesse auf localhost ausgeführt wurden, lag das Maximum bei $n = 485$. Der begrenzende Faktor war hier vor allem die Ressourcen des Betriebssystems. Jeder Knoten lief als eigener Prozess mit einem eigenen UDP-Socket. Dadurch waren die Anzahl gleichzeitiger Prozesse und die Anzahl offener Sockets beschränkt. In Aufgabe 02, wo echte verteilte Computer gefordert waren, ist das maximale n durch die Anzahl der tatsächlich verfügbaren Rechner begrenzt gewesen. Der Versuch, den ganzen CIP-Pool zum Laufen zu bringen, scheiterte leider an den Firewall-Einstellungen der Uni, an denen ohne Administrator-Zugriff keine Änderungen gemacht werden konnten. Deswegen beschränkte sich das maximale n auf die zuhause verfügbaren Computer, $n = 3$. In Aufgabe 03, wo der Simulator genutzt wurde, konnte ein n von 16384 mit 8GB Arbeitsspeicher für die JVM erreicht werden. Da der Simulator virtuelle Threads nutzt, ist die Thread-Anzahl kein Limit, sondern der begrenzende Faktor war der Heap-Speicher. Der Simulator kann kein echtes Multicast, sondern verschickt bei einem `broadcast()` intern $(n - 1)$ einzelne Nachrichten. Auch hier, um die Vergleichbarkeit zu den anderen Aufgaben zu erhalten, wurde mit $p = 1$ gestartet. Dadurch zünden in der ersten Runde alle eine Rakete, was bei $n = 16384$ schon mehrere hundert Millionen Nachrichten sind. Der Lauf mit $n = 16384$ dauerte insgesamt 2 Stunden und 48 Minuten. Im Verlauf wurde die Simulation schneller, da p nach jeder Runde kleiner wurde und dadurch immer weniger Raketen gezündet wurden. Ein noch größerer Lauf mit $n = 32768$ wäre eventuell mit einer niedrigeren Startwahrscheinlichkeit möglich gewesen. Mit $p = 1$ war dieser Lauf jedoch nicht mehr praktikabel.

Bei den Rundenzeiten liegen Aufgabe 01 und Aufgabe 03 für kleinere und mittlere n zunächst in einer ähnlichen Größenordnung. Bei großen Ringgrößen steigen die Zeiten in Aufgabe 03 jedoch stark an. Ein Grund dafür ist die unterschiedliche Umsetzung des Broadcast. Bei echtem UDP-Multicast sendet der Prozess nur ein Paket an die Multicast Gruppe. Im Simulator wird ein Broadcast aber intern als Zustellung an alle anderen Knoten umgesetzt. Pro gezündeter Rakete entstehen $n-1$ einzelne Nachrichten. Dadurch steigen die Broadcast

Kosten im Simulator linear mit der Anzahl der Knoten. Die Simulation kann also höhere Kosten eines Broadcast bei großen n haben als die reale UDP Multicast Variante. Die maximale Rundenzeit bei $n = 16384$ lag bei 3.867.439 ms. Das entspricht ungefähr 1 Stunde und 4 Minuten.

n	01 Multicasts	01 Runden	01 avg ms	03 Multicasts	03 Runden	03 avg ms
2	4	6	2.41	5	9	1
16	28	8	3.46	34	12	2.5
128	255	11	19.95	246	11	29.45
256	544	12	57.18	537	12	99.25
1024	-	-	-	2024	12	1388.92
8192	-	-	-	16500	18	59069.67
16384	-	-	-	32684	17	464319.94

In Aufgabe 02 lagen die Rundenzeiten bereits bei $n = 3$ bei durchschnittlich rund 100 ms. Das liegt daran, dass die Nachrichten hier nicht nur lokal auf einem Rechner verarbeitet wurden, sondern tatsächlich über ein Netzwerk zwischen mehreren physischen Maschinen übertragen werden mussten. Dadurch kommen reale Netzwerk Latenzen, unterschiedliche Rechnerleistungen und mögliche Verzögerungen durch das Betriebssystem hinzu.

Vom Experimentalaufwand her war Aufgabe 02 am aufwendigsten, da das Skript und die Konfigurationsdatei an die physischen Rechner verteilt werden musste. Bei den Aufgaben 01 und 03 war der Experimentalaufwand nicht so hoch, weil die Skripte automatisiert gestartet werden konnten.

Aufgabe 04- Konsistenz

Nachdem die ersten drei Aufgaben die Funktionalität der verteilten Anwendung gezeigt haben, geht es in Aufgabe 04 um die Frage, ob unter bestimmten Umständen Inkonsistenzen auftreten können. Bevor man prüfen kann, ob etwas konsistent ist, muss festgelegt werden, was Konsistenz in diesem Kontext bedeuten soll. Als Orientierung dienen vier grundlegende Fragen, die man an jedes Nachrichtensystem stellen kann:

1. Kommt jede gesendete Nachricht an oder gehen welche auf dem Weg verloren?
2. Kommen Nachrichten in der Reihenfolge an, in der sie gesendet wurden?
3. Erreicht ein Broadcast alle Empfänger gleichzeitig?
4. Wissen alle Knoten dasselbe zu selben Zeit?

Auf Basis dieser Fragen lassen sich drei Konsistenzkriterien ableiten, die für die Anwendung sinnvoll sind.

- Jede vor der Terminierung gezündete Rakete soll bei den anderen Knoten ankommen. Es gilt: die Summe der empfangenen Raketen entspricht der Summe der gesendeten Raketen $\cdot (n-1)$. Der Faktor $n - 1$ ergibt sich daraus, dass ein Knoten seine eigene Rakete nicht per Broadcast empfängt.
- Alle Knoten sollen die Raketen in derselben Reihenfolge sehen.

- Wenn Rakete A kausal vor Rakete B liegt (A vor B gezündet), muss jeder Knoten A vor B sehen (Thema Lamport Zeit aus der Vorlesung).

Um zu prüfen, ob die definierten Kriterien tatsächlich verletzt werden können, wurde ein Blick in den Quellcode des Simulators geworfen. In `Network.class` ist die das Senden eines broadcast mit einer einfachen for-Schleife umgesetzt. Dementsprechend wird die Nachricht nacheinander an jeden einzelnen Knoten zugestellt. Während die Rakete bei den ersten Knoten schon angekommen ist, ist sie bei anderen noch nicht angekommen und in dieser Zwischenzeit können andere Ereignisse passieren. Genau hier könnten Reihenfolge-Inkonsistenzen oder Nachrichtenverluste bei der Terminierung entstehen.

In der Klasse `Mailbox` ist die `receive()` Methode implementiert. Für eine Mailbox würde man eigentlich eine Queue erwarten, bei der die älteste Nachricht zuerst verarbeitet wird. Im Simulator wird die Mailbox jedoch mit einer einfachen `ArrayList` umgesetzt. Außerdem wird nicht immer automatisch die älteste Nachricht verarbeitet. Welche Nachricht als nächstes entnommen wird, entscheidet die Methode `selectMessageInQueue`. Diese Methode aus `SimulationBehavior.java` gibt im Standardfall zwar 0 zurück, womit sich der Simulator standardmäßig wie eine FIFO-Queue verhält, aber sobald eine Verteilungsfunktion gesetzt wird, wird ein zufälliger Index gewählt. Der Simulator ist also darauf ausgelegt Inkonsistenzen auslösen zu können.

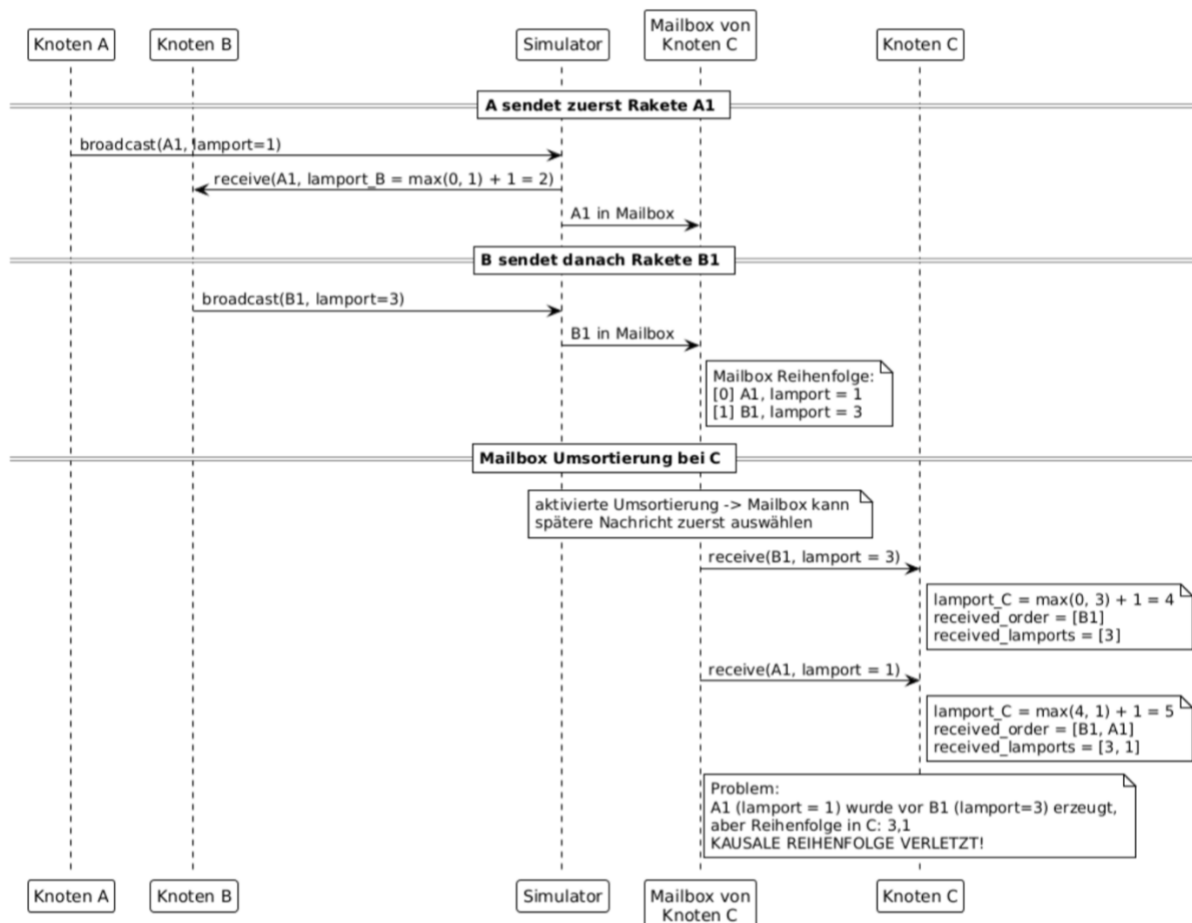
Im Simulator gibt es keine gemeinsame globale Zeit und keinen Mechanismus, der die kausale Ordnung von Nachrichten zwischen Knoten garantiert. Auch gibt es keinen gemeinsamen Rundenbegriff. Im Algorithmus zählt nur Knoten 0 die Runden, die übrigen Knoten reichen den Token nur weiter und haben im Prinzip keine Vorstellung davon, in welcher Runde sie sich gerade befinden. Dadurch könnte ein Knoten beispielsweise eine Rakete empfangen, deren Sender bereits in einer späteren logischen Runde war als der Empfänger selbst.

Im umgesetzten Code zu Aufgabe 03 lassen sich damit diese Konkreten Schwachstellen festmachen:

- Beim Aufruf von `broadcast()` wird angenommen, dass alle anderen Knoten die Rakete zur selben Zeit sehen, was aber nicht gilt
- Beim Empfang wird angenommen, dass Nachrichten in einer sinnvollen Reihenfolge ankommen, was zwischen verschiedenen Sendern nicht garantiert ist.
- Beim return im `Terminate`-Fall werden alle noch in der Mailbox liegenden Raketen verworfen, was im schlimmsten Fall zu Nachrichtenverlust führt
- Der Rundenbegriff wird nur von Knoten 0 verwaltet, es existiert kein gemeinsames Verständnis.

Um diese potenziellen Inkonsistenzen zu erkennen und evtl. zu vermeiden, wurden folgende Anpassungen vorgenommen. Jeder Knoten zählt mit, wie viele Raketen er gesendet (`myMulticasts`) und empfangen (`fireworksReceived`) hat. Nach Ende der Simulation wird berechnet, ob die Zahl der empfangenen Raketen stimmt. Damit die Reihenfolge der empfangenen Raketen vergleichbar wird, bekommt jede Rakete eine eindeutige ID aus Sender und laufender Nummer. Jeder Knoten speichert die IDs in Empfangsreihenfolge in einer Liste. Nach Ende der Simulation werden die Listen aller Knoten mit der von Knoten 0 verglichen. Außerdem wurde eine Lamport-Uhr geführt. Vor jedem Senden wird der lokale

Zähler erhöht und der neue Wert mit der Nachricht verschickt. Bei jedem Empfang wird $\text{lamport} = \max(\text{eigene_lamport}, \text{empfangener_lamport}) + 1$ gesetzt. So gilt: Wenn Ereignis A kausal vor B liegt, ist $\text{timestamp}(A) < \text{timestamp}(B)$. Sowohl Token als auch Firework tragen einen Lamport-Zeitstempel mit. Beim Empfang einer Rakete wird der Zeitstempel zusammen mit der Rakete in die Empfangsliste eingetragen. Nach Ende der Simulation prüft jeder Knoten seine Empfangsliste auf aufsteigende Zeitstempel. Wird eine Rakete mit kleinerem Zeitstempel NACH einer mit größerem Zeitstempel empfangen, liegt eine Verletzung der kausalen Ordnung vor.



Die Erkennungsmechanismen wurden an einem Ring mit $n = 512$ Knoten und den Parametern $p = 1$, $k = 3$ getestet. Dabei wurden zwei Konfigurationen verglichen: einmal der Standardmodus des Simulators und einmal mit aktivierter Mailbox-Umsortierung.

Ergebnis im Standardmodus:

Konsistenz-Check: erwartet=529907, empfangen=529907, verloren=0

Reihenfolge-Check: 512 von 512 Knoten haben dieselbe Reihenfolge wie Knoten 0

Kausal-Check: 0 Verletzungen über alle Knoten

Ergebnis bei Umsortierung:

Konsistenz-Check: erwartet=537572, empfangen=537572, verloren=0

Reihenfolge-Check: 356 von 512 Knoten haben dieselbe Reihenfolge wie Knoten 0

INKONSISTENZ: 156 Knoten sehen Raketen in anderer Reihenfolge

Kausal-Check: 156 Verletzungen über alle Knoten

Die Zustellung bleibt vollständig, aber sowohl die Reihenfolge-Konsistenz als auch die kausale Konsistenz werden verletzt. Die Zustellung bleibt stabil, weil das Token seriell durch den Ring wandert und das Terminate erst losgeschickt wird, nachdem drei Runden lang niemand mehr gezündet hat. In dieser Zeit haben alle Raketen Gelegenheit, in den Mailboxen anzukommen und verarbeitet zu werden. Auch das Terminate selbst läuft seriell durch den Ring und braucht n Schritte. Broadcast-Raketen liegen meist vor ihm in den Mailboxen und werden zuerst verarbeitet.

Insgesamt zeigen die Aufgaben, dass die reine Umsetzung der verteilten Anwendung nur ein Teil des Problems ist, da besonders Nachrichtenreihenfolge, Zustellung und Konsistenz eine wichtige Rolle spielen.